Corte del proyecto

Estado de SABIA al 28 de octubre de 2026

PROYECTO

SABIA — Sistema de Automatización, Base de información e Inteligencia Artificial

ÁREA

Calidad
Capital Salud EPS-S

DOCUMENTO

04 · Documentación de traspaso

FECHA DE CORTE

28 de octubre de 2026

REPOSITORIO

faazdy/almera-docs
rama master

RESPONSABLE

Leonardo Rengifo Ramos

---

00 Alcance del documento

Estado de SABIA a esta fecha: qué tiene, para qué se hizo, cómo está construido y qué falta. Es un documento de corte: describe el proyecto **tal como está hoy**.

El detalle de cada parte vive en sus propios documentos, que este corte referencia en lugar de repetir.

DOCUMENTOS RELACIONADOS

TEMA	DOCUMENTO
Skills (8 de módulos Almera + 2 de entregables)	`docs/01-skills.md`
Informes / dashboards	`docs/02-informes.md`
Documentación de módulos de Almera	`docs/03-almera.md`
Asistente RAG y ETL (operación)	`rag/README.md`
Rutas del sitio y menú compartido	`routes.md`

01

Resumen

SABIA (Sistema de Automatización, Base de información e Inteligencia Artificial) es el proyecto del área de Calidad de Capital Salud para que **nadie tenga que volver a ver una grabación de una hora para resolver una duda de Almera**, y para automatizar entregables repetitivos del área.

EN CIFRAS

PIEZAS	MÓDULOS ALMERA	SKILLS DE CLAUDE	DASHBOARDS
5	8	10	2

PIEZAS DEL PROYECTO

PIEZA	QUÉ ES	ESTADO
Documentación de Almera	Guías web + GuIA `.md` de 8 módulos	● **Completa** Algunos módulos tienen dudas abiertas con soporte
Skills de Claude	8 skills de módulo (responden dudas) + 2 de entregables (presentaciones, diagramas de flujo)	● **Funcionales** `diagramas-flujo` con fuentes desincronizadas
Informes	2 dashboards que leen el listado maestro en el navegador	● **Funcionales**
Asistente “Pregúntale a SABIA” NUEVO	Chat web con RAG-lite sobre la documentación, vía OpenRouter	● **Funcional en local** Falta publicarlo en el servidor del área
ETL de la carpeta compartida NUEVO	Convierte Word/PDF/PPT/Excel que sube el área en conocimiento para el asistente	● **Funcional en local** Falta conectarlo a la carpeta de red

LO QUE CAMBIÓ EN ESTE CORTE

SABIA dejó de ser solo un sitio estático. Ahora tiene un **servidor** (`rag/`) que publica el sitio y el asistente en una sola dirección, un **ETL** para que personas no técnicas alimenten el conocimiento y un **menú nuevo** (diseño 1c) en todas las páginas.

02 Para qué se hizo

EL PROBLEMA

A	La única fuente para resolver una duda de Almera eran **grabaciones de capacitación de 1 a 3 horas**, de las que dos minutos respondían la pregunta.
B	Casi nadie usa todos los módulos, pero todos dependían de la misma grabación genérica.
C	Las tareas repetitivas del área (presentaciones institucionales, diagramas en Bizagi, revisar el listado maestro) dependían de una o dos personas.

LOS OBJETIVOS

01	**Autoservicio.** Que cualquiera resuelva su duda de Almera en segundos, con la ruta exacta y los pasos documentados.
02	**No inventar.** Si algo no está documentado o no está confirmado con soporte, el sistema lo dice en vez de suponer.
03	**Conocimiento vivo.** Que el área pueda agregar y actualizar conocimiento sin depender de un desarrollador.
04	**Automatizar entregables** siguiendo los lineamientos institucionales.

POR QUÉ UN ASISTENTE PROPIO, SI YA EXISTÍAN LAS SKILLS

Las skills solo funcionan para quien tiene **cuenta de Claude** y sabe instalarlas. El asistente web abre SABIA a **cualquier persona de la entidad desde el navegador**, sin cuenta ni instalación.

Las skills siguen siendo útiles para quien trabaja en Claude, y las dos usan la misma documentación.

---

03 Qué tiene el proyecto hoy

3.1 Documentación de Almera `almera/`

8 módulos, cada uno en **tres copias del mismo contenido** que se sincronizan a mano: guía web (`index.html`), GuIA (`AI/*.md`) y skill (`AI/*.skill`).

MÓDULOS POR GRUPO

GRUPO	MÓDULOS
Documentos	Gestión documental, Mapa de procesos (incluye SNC), Mecanismos de integración, Modelos de gestión
Evaluaciones	Evaluaciones / Auditorías, Indicadores, Planes de mejora
Proyectos	Proyectos

Más: árbol de navegación de Admin (`assets/almera-structure.md`), códigos de estructura organizacional y la guía para instalar las skills (`almera-IA/`). Detalle y pendientes en `03-almera.md`.

3.2 Skills de Claude

8	**Skills de módulo** (`almera-*`): responden dudas con la GuIA del módulo.
2	**Skills de entregables:** `presentaciones-institucionales` (PPTX/HTML/PDF con la plantilla y paleta institucional) y `diagramas-flujo` (BPMN/PDF/PNG/JPG/SVG estilo Bizagi, con validación por colores).

Detalle en `01-skills.md`.

3.3 Informes (dashboards)

INFORME	RUTA	QUÉ MUESTRA
Informe de documentos Almera	`almera/informe-documentos/`	KPIs y calidad de datos del export de Almera
Control de documentos del SGC	`almera/control-documentos-sgc/`	Vigencia y actualización del listado maestro del SGC

Los dos procesan el Excel **en el navegador** (nada sale del equipo) y permiten indicadores personalizados sin código. Detalle en `02-informes.md`.

3.4 Asistente “Pregúntale a SABIA” `rag/` NUEVO

— Chat en `/asistente/`, con la identidad visual de la portada (DM Sans, JetBrains Mono, fondo hueso, acento naranja).

— Responde con la **ruta exacta y los pasos**, avisa cuando algo está “pendiente de validar con soporte” y cita la fuente (“Fuente: Indicadores › Gestión de metas y semaforización”).

— Muestra en qué módulo buscó y qué secciones consultó. Tiene un selector para fijar el módulo a mano.

— Respuesta en *streaming* (va apareciendo mientras se escribe).

3.5 ETL de la carpeta compartida `rag/app/etl.py` NUEVO

— El área guarda un Word, PDF, PowerPoint, Excel o `.txt` en la subcarpeta de su módulo.

— El servidor revisa la carpeta cada 5 minutos, lo convierte a Markdown limpio, le aplica un **semáforo** (🟢🟡🔴) y lo suma al asistente **sin reiniciar**.

— Deja un `ESTADO SABIA.txt` en la misma carpeta, con el resultado y qué corregir.

— La carpeta modelo (`rag/carpeta-compartida/`) trae `LEEME.txt`, un prompt para ordenar documentos con IA y una plantilla de Word.

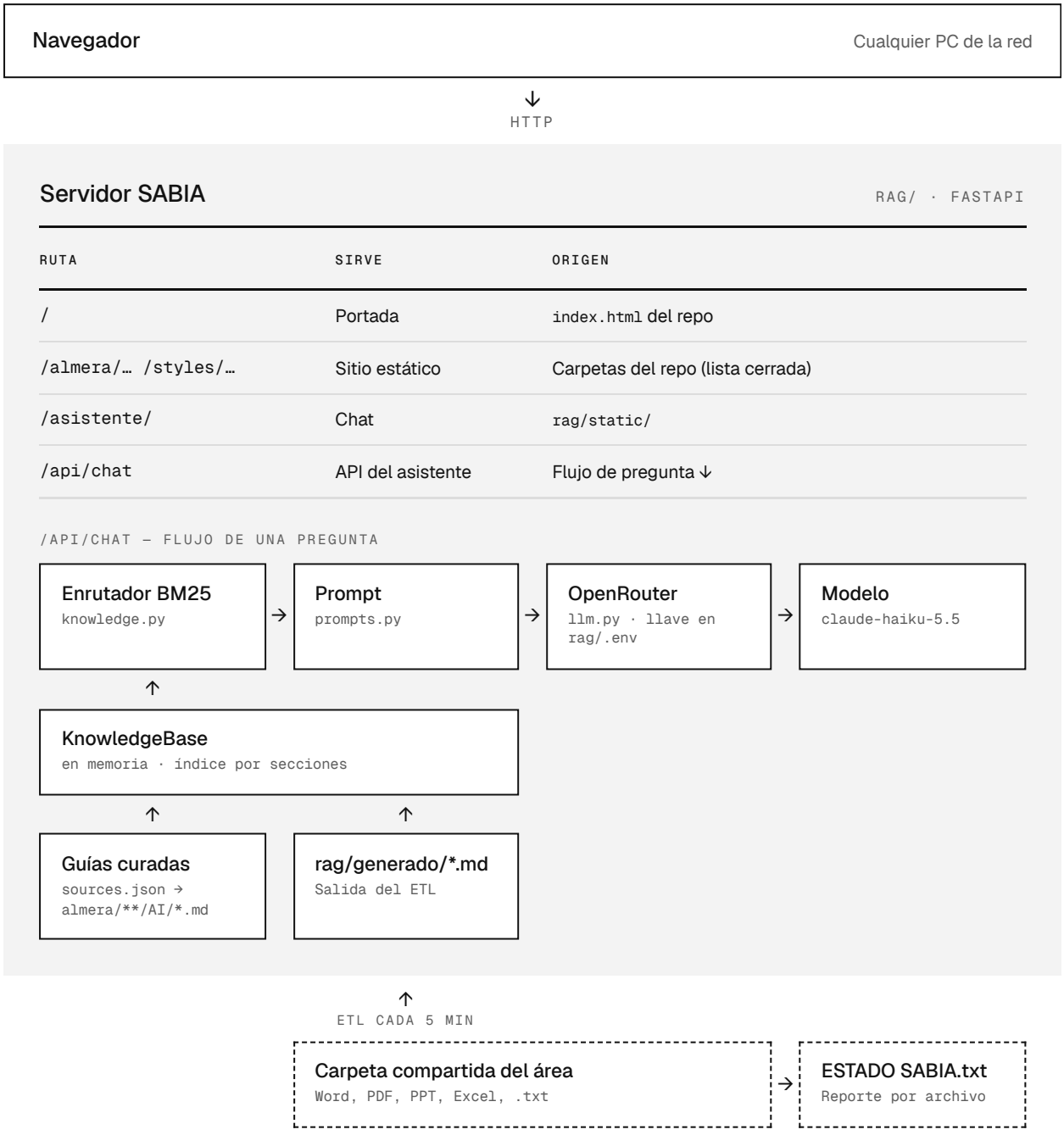
3.6 Sitio y navegación

— Portada (`index.html`) con 4 entradas: Almera, Agentes de IA, Control de documentos SGC y **Pregúntale a SABIA** (nueva).

— **Menú 1c** en todas las páginas (salvo los dos informes, por diseño): un círculo de 44 px que abre un menú oscuro a pantalla completa. Un solo archivo, `almera/assets/nav.js`, que se monta solo en el header de cada página. Detalle en `routes.md`.

04 Arquitectura

4.1 Vista general



4.2 Dos formas de servir el sitio

MODO	CÓMO	QUÉ FUNCIONA
Servidor SABIA Recomendado	`rag\iniciar.ps1` → `http://127.0.0.1:8800/`	Todo: sitio, menú, asistente y ETL
Solo estático	`npx serve`, GitHub Pages o doble clic al HTML	Todo menos el asistente (su enlace da 404)

Funciona en los dos modos porque **todos los enlaces son relativos** y ninguno lleva puertos.

4.3 Qué pasa en cada pregunta

01 **Enrutar** `knowledge.py`

- Busca la pregunta, más la anterior para los seguimientos, en un índice **BM25** de todas las secciones (partidas por títulos `#`–`###`).
- Elige el módulo con mejor puntaje, y un segundo si saca al menos el 60 % del primero.

02 **Armar el prompt** `prompts.py`

- Lleva las reglas, el árbol de navegación de Admin y **los documentos completos** del módulo.
- Si el módulo no cabe en `MAX_CONTEXT_CHARS`, van primero sus documentos más relevantes.

03 **Llamar al modelo** `llm.py`

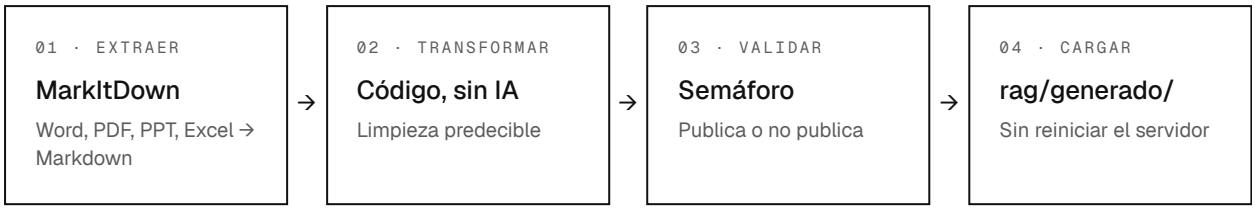
- POST a OpenRouter desde el servidor con *streaming*, temperatura 0,2 y `data_collection: deny`.

04 **Responder** `main.py` → `static/app.js`

- Eventos SSE en orden: `meta` (módulo y fuentes), luego `delta` (texto) y al final `done`.
- El navegador pinta el Markdown con `marked` y lo limpia con `DOMPurify`.

> IMPORTANTE
>
> El modelo **no lee archivos ni recuerda nada**: en cada pregunta el servidor le pega el texto relevante. El historial lo guarda el navegador (últimos 8 mensajes).

4.4 Qué hace el ETL



TRANSFORMAR

— Quita imágenes y números de página.

— Quita encabezados y pies repetidos, solo en PDF y PPT.

— Convierte en títulos los párrafos que son solo negrita.

— Limpia los `NaN` de Excel.

VALIDAR

NO PUBLICA	PUBLICA CON AVISO	PUBLICA
Escaneado o casi vacío, transcripción de reunión, texto desarmado, largo sin títulos, formato antiguo, carpeta inválida o archivo suelto.	El documento entra al asistente y el reporte indica qué conviene corregir.	El documento entra al asistente sin observaciones.

CARGAR

— El `.md` va a `rag/generado/` y el reporte a la carpeta compartida.

— **Nunca mueve ni modifica los originales.**

— Solo reprocesa los archivos que cambiaron.

4.5 Mapa del repositorio

```
almera-docs/
├── index.html                Portada de SABIA
├── routes.md                 Rutas del sitio y menú (nuevo)
├── docs/                     Documentación de traspaso (este archivo)
├── styles/                   Estilos de la portada
├── almera/                   Documentación de Almera, informes y nav.js
│   └── assets/nav.js         Menú compartido (diseño 1c)
├── calidad-ai-agents/        Skills de entregables
└── rag/                      Servidor SABIA (nuevo)
    ├── app/
    │   ├── main.py           Servidor: sitio + chat + API + ciclo del ETL
    │   ├── knowledge.py      Base de conocimiento, índice BM25, enrutamiento
    │   ├── etl.py            Carpeta compartida → Markdown con semáforo
    │   ├── llm.py            Cliente de OpenRouter
    │   ├── prompts.py        Reglas del asistente
    │   └── config.py         Lee rag/.env
    ├── static/               Interfaz del chat
    ├── carpeta-compartida/   Modelo de la carpeta del área (LEEME, prompt, plantilla)
    ├── sources.json          Qué guías curadas forman cada módulo
    ├── .env.example          Plantilla de configuración (el .env real no se sube)
    └── iniciar.ps1           Arranque en Windows
```

05 Stack tecnológico

CAPA	TECNOLOGÍA	PARA QUÉ
Frontend	HTML, CSS y JavaScript plano (sin framework, sin build)	Sitio, informes, chat y menú
	DM Sans + JetBrains Mono (Google Fonts)	Tipografía de SABIA
	Chart.js, SheetJS (`vendor/`)	Gráficos y lectura de Excel en los informes
	marked 12.0.2, DOMPurify 3.1.6 (`rag/static/vendor/`)	Markdown seguro en el chat
Backend	Python 3.13	Lenguaje del servidor
	FastAPI 0.115 + Uvicorn 0.34	Servidor web, API y archivos estáticos

CAPA	TECNOLOGÍA	PARA QUÉ
	httpx 0.28	Llamadas a OpenRouter con *streaming*
	python-dotenv 1.1	Configuración y llave en `rag/.env`
	truststore 0.10	Usa los certificados de Windows (la red inspecciona TLS)
	MarkItDown 0.1.9 (Microsoft)	Word/PDF/PPT/Excel → Markdown en el ETL
IA	OpenRouter	Acceso a modelos; se cambia el modelo sin tocar código
	`anthropic/claude-haiku-4.5` (por defecto)	Modelo del asistente: barato, buen español, sigue reglas
	Claude Agent Skills (`.skill`)	Las 10 skills del proyecto
Búsqueda	BM25 propio en Python puro (`rag/app/knowledge.py`)	Enrutar preguntas sin base vectorial
Datos	Archivos Markdown en el repo y en la carpeta compartida	No hay base de datos
Herramientas	Git + GitHub (`faazdy/almera-docs`), Claude Code, Claude Design, `npx serve`	Desarrollo, diseño y pruebas

> LO QUE DELIBERADAMENTE NO SE USÓ
>
> Base de datos, base vectorial, embeddings, framework de frontend, LangChain u otro framework de LLM, Docker. El volumen actual no los justifica y cada uno suma mantenimiento (ver punto 06).

06 Cómo se hizo: decisiones y por qué

DECISIÓN	POR QUÉ	CUÁNDO REVISARLA
RAG-lite Enrutar + inyectar el módulo completo, en vez de RAG clásico con embeddings	Todo el conocimiento son ~240 KB (~65k tokens). Cada módulo cabe entero en el contexto. Partir en fragmentos corta los procedimientos paso a paso, que es el peor error posible en este dominio.	Si el conocimiento pasa de ~150k tokens o de cientos de documentos. El punto de cambio es `KnowledgeBase.route()`.

DECISIÓN	POR QUÉ	CUÁNDO REVISARLA
BM25 léxico para enrutar	Sin dependencias, sin costo, determinista y fácil de depurar. Acertó en las preguntas de prueba de cada módulo.	Si el enrutador se equivoca seguido con preguntas reales.
OpenRouter en vez de un proveedor fijo	Permite cambiar de modelo con una línea del `.env` y comparar costo y calidad.	—
Un solo servidor para sitio + chat	Con dos servidores, los enlaces entre sitio y chat necesitaban puertos fijos y se rompían si uno estaba apagado.	—
Lista cerrada de carpetas publicadas `SITE_DIRS`	Montar la raíz del repo expondría `rag/.env` (la llave) y `.git`. Se verificó que todo lo demás responde 404.	Al agregar una carpeta de primer nivel.
ETL combinado Código para convertir y validar + prompt de IA para lo que no se puede limpiar	El código es predecible y nunca altera un procedimiento. La IA sí puede convertir una transcripción en guía, pero con revisión humana.	Si el volumen de documentos “en rojo” crece: evaluar limpieza con LLM dentro del ETL, con aprobación.
El ETL no mueve ni edita los originales	Mover archivos en una carpeta de red que otros usan genera confusión y conflictos con archivos abiertos.	—
Revisión periódica (polling) En vez de detección instantánea	Las notificaciones de cambios no son confiables en carpetas de red, y así se evita procesar archivos a medio copiar.	Bajar a 60 s si se quiere respuesta más rápida (no cuesta nada).
Menú en un solo `nav.js` que se monta solo	Copiar el HTML del menú en 16 páginas obliga a mantener 16 copias. Así, las 12 páginas de módulo lo reciben sin editar su HTML.	—
`truststore` en vez de desactivar SSL	La red corporativa inspecciona TLS con un certificado propio. Desactivar la verificación sería inseguro.	—

CÓMO SE CONSTRUYÓ ESTE CORTE

Se hizo en una sesión de trabajo con Claude Code, por fases acordadas:

01 **Evaluación:** ¿vale la pena un RAG? Sí, pero “lite”, por el volumen.

02 **Asistente RAG-lite** con OpenRouter y la llave en `.env`.

03	**Fase 1:** identidad visual de SABIA en el chat.
04	**Menú 1c** importado desde Claude Design y adaptado a todas las páginas.
05	**Un solo servidor** para eliminar los puertos fijos en los enlaces.
06	**Fase 3:** ETL de la carpeta compartida y `routes.md`.

Todo se probó en el navegador (Playwright + Edge, escritorio y celular) y con documentos reales del repo. Commits en `master`: `4209d1e`, `b01857a`, `9b268e6`, `ac0f4cf`, `b9d1590`.

07 Seguridad

A	**Llave de OpenRouter.** Solo en `rag/.env`, que está en `.gitignore`. Nunca llega al navegador. Se verificó que no está en el historial de Git.
B	**Qué publica el servidor.** Solo `index.html`, `styles/`, `almera/` y `calidad-ai-agents/`. Lo demás da 404, incluidos los intentos con `../` codificado.
C	**Privacidad de las preguntas.** `data_collection: deny` hace que OpenRouter solo use proveedores que no guardan ni entrenan con los datos. Aun así, la regla es **no pegar datos personales ni de pacientes** (ver `calidad-ai-agents/SEGURIDAD.md`).
D	**Respuestas.** El Markdown del modelo se sanitiza con DOMPurify antes de mostrarse.
E	**Límites.** Preguntas de máximo 4000 caracteres y máximo 8 mensajes de historial.
F	**Carpeta compartida.** Lo que se guarde ahí termina en las respuestas. Solo deben tener permiso de escritura quienes publican.

> ATENCIÓN · EL ASISTENTE NO TIENE AUTENTICACIÓN
>
> Mientras escuche en `127.0.0.1` solo lo usa ese equipo. Si se abre a la red (`HOST=0.0.0.0`), cualquiera en la red podría usarlo y gastar créditos. Hay que configurar un **límite de gasto** en la llave de OpenRouter.

08 Operación y costos

ARRANCAR

```
cd rag
.\iniciar.ps1          # la primera vez crea el entorno e instala dependencias
```

Hay que poner la llave en `rag/.env` antes del primer arranque.

CONFIGURACIÓN · RAG/.ENV

VARIABLE	POR DEFECTO	QUÉ HACE
`OPENROUTER_API_KEY`	–	Llave de OpenRouter
`OPENROUTER_MODEL`	`anthropic/claude-haiku-4.5`	Modelo del asistente
`OPENROUTER_DENY_DATA_COLLECTION`	`true`	Solo proveedores que no guardan datos
`HOST / PORT`	`127.0.0.1 / 8800`	Dónde escucha el servidor
`MAX_CONTEXT_CHARS`	`120000`	Tope de documentación por pregunta
`KNOWLEDGE_DIR`	`carpeta-compartida`	Carpeta del ETL. En producción, `\\servidor\Calidad\SABIA` **sin comillas**
`ETL_INTERVAL_SECONDS`	`300`	Cada cuánto se revisa la carpeta

COSTO ESTIMADO · HAIKU 5.5

POR PREGUNTA	1.000 PREGUNTAS/MES	TOKENS DE ENTRADA	PRECIO DE ENTRADA
US$0,002	≈ US$2	8k – 17k por pregunta	US$0,10 por millón de tokens

PROBAR EL ETL A MANO · DESDE RAG/

```
.venv\Scripts\python -m app.etl
```

09 Pendientes y riesgos

PARA DEJAR EL ASISTENTE EN PRODUCCIÓN · FASE 2

- ☐ **Definir con TI dónde corre el servidor:** un equipo siempre encendido y con acceso a la carpeta de red.
- ☐ **Permisos de la carpeta:** el servidor solo lee y escribe `ESTADO SABIA.txt`. Las personas que publican tienen escritura en las subcarpetas.
- ☐ **Abrir el puerto 8800** en la red interna si los compañeros van a entrar desde su PC.
- ☐ **Autenticación básica** antes de abrirlo a la red (hoy no tiene).
- ☐ **Límite de gasto** en la llave de OpenRouter.
- ☐ **Copiar `rag/carpeta-compartida/`** al servidor y apuntar `KNOWLEDGE_DIR` a esa ruta.
- ☐ **Validar la calidad de las respuestas** con un set de preguntas reales del área por módulo. Durante el desarrollo se probaron el enrutamiento, la navegación y el ETL, no la calidad de las respuestas del modelo.

MEJORAS IDENTIFICADAS

☐	Revisar la carpeta cada **60 s** en vez de 300 (sin costo).
☐	Incluir las notas del semáforo dentro de los `.md` generados, además de `ESTADO SABIA.txt`.
☐	Registrar las preguntas que el asistente no pudo responder, para saber qué documentar.
☐	Pruebas automatizadas del ETL y del enrutador. Hoy no hay: las pruebas de este corte fueron manuales.

RIESGOS Y DEUDAS QUE VIENEN DE ANTES · VER 01-03

A	**`diagramas-flujo`.** El `.skill` es la única copia completa del generador; las fuentes del repo están desactualizadas (`01-skills.md` §3.4).
B	**Tres copias por módulo de Almera** (guía web, GuIA, skill) que se sincronizan a mano.
C	**Dudas abiertas con soporte de Almera** en varios módulos (`03-almera.md`).
D	**Documentos desactualizados frente a este corte.** El `README.md` raíz y `sabia.MD` todavía describen SABIA como sitio 100 % estático, sin servidor.
E	**Los `.docx` del repo son transcripciones de reuniones,** no guías. El ETL los marca en rojo, correctamente. Si se quiere su contenido, hay que pasarlos por el prompt de limpieza.

10 Próximos pasos sugeridos

Lo primero es publicar el servidor en la red del área con TI; el resto depende de tener usuarios reales usando el asistente.

01	**Fase 2:** publicar el servidor en la red del área con TI (punto 09).
02	**Piloto con usuarios reales** de uno o dos módulos, registrando las preguntas sin respuesta.
03	**Reconciliar `diagramas-flujo`** (riesgo de perder el generador).
04	**Actualizar el README raíz y `sabia.MD`** con el asistente y la arquitectura de servidor.
05	Ideas del `to-do`: skills para ChatGPT Pro, agente local con LM Studio y **RAG para la DEP**. Este último puede reutilizar casi todo `rag/`: cambian `sources.json`, la carpeta compartida y el prompt.

11

Glosario

TÉRMINO	SIGNIFICADO
RAG	*Retrieval-Augmented Generation*: buscar la información relevante y dársela al modelo junto con la pregunta, para que responda con datos reales y no de memoria.
RAG-lite	La variante de SABIA: en vez de buscar fragmentos, elige el módulo y le pasa su documentación completa.
BM25	Algoritmo clásico de búsqueda por palabras clave (el de los buscadores tradicionales).
ETL	*Extract, Transform, Load*: extraer el contenido de un archivo, limpiarlo y cargarlo donde se usa.
GuIA	El documento `.md` de referencia de cada módulo de Almera.
Skill	Paquete `.skill` que Claude carga cuando el pedido lo requiere.
OpenRouter	Servicio que da acceso a muchos modelos de IA con una sola llave.
Streaming / SSE	La respuesta llega por partes mientras se genera, en vez de esperar a que termine.
Semáforo	🟢 publicado · 🟡 publicado con advertencias · 🔴 no publicado